

Projektarbeit MLOps

Wir nutzen die Projektarbeit, um die Themen FTI-Architektur und Featurestore zu vertiefen. Dazu sollen drei Pipelines implementiert werden. Als Featurestore verwenden wir den Hopsworks Featurestore, welcher nach Erstellung eines Accounts gratis verwendet werden kann. Als Model-Registry kann auch Hopsworks verwendet werden, MLFlow oder das Modell wird lediglich lokal gespeichert, was im Rahmen der Projektarbeit auch zulässig ist.

Die Projektarbeit kann auf Github Codespaces (gestartet aus einem eigenen Github Repo) oder lokal auf dem Laptop umgesetzt werden.

Wiederum geht es nicht darum, ein performantes Modell zu erstellen. Accuracy, F1-Score usw. des trainierten Modells sind also bei dieser Projektarbeit nicht wichtig.

Ablauf

1. Wahl einer geeigneten Datenquelle
2. Auswahl eines Targets (was soll vorhergesagt werden) und passender Features
3. Erstellen eines Github Repositorys
4. Erstellen eines Hopsworks accounts auf hopsworks.ai
5. Installation der Dependencies (Hopsworks verlangt Python <3.14)
6. Implementation der Feature-Pipeline
7. Implementation der Trainings-Pipeline
8. Implementation der Inferenz-Pipeline
9. Dokumentation in einem README.md File im Repository

Datenquellen

Es kann entweder eine öffentliche Datenquelle verwendet werden, oder es werden eigene Daten verwendet. Wichtig dabei ist, dass die Daten unterteilbar sind in historische Daten für das Modell-Training und aktuelle Daten für die Inferenz. Wenn die Datenquelle keine aktuellen Daten zur Verfügung stellt, müssen diese simuliert werden, entweder künstlich durch einen Generator wie in den Übungen, oder (einfacher) durch Abspalten eines Teils des Trainingssets.

Öffentliche Datenquellen

Hier sind einige öffentliche Datenquellen als Inspiration:

Data Source	URL
Open-Meteo Weather API	https://open-meteo.com/
TrafikLab (Swedish Transit/GTFS)	https://www.trafiklab.se/
Spotify Web API	https://developer.spotify.com/documentation/web-api
NOAA Climate Data	https://www.ncdc.noaa.gov/cdo-web/webservices/v2

Data Source	URL
TomTom Traffic API	https://developer.tomtom.com/
TMDB (The Movie Database)	https://developer.themoviedb.org/docs
Riot Games API	https://developer.riotgames.com/
ArXiv API	https://info.arxiv.org/help/api
EU DSA Transparency Database	https://transparency.dsa.ec.europa.eu/
GitHub REST API	https://docs.github.com/en/rest

Features

Es muss mindestens ein aggregiertes Feature (über mehrere Timesteps oder über mehrere Datenquellen) und ein aktuelles Feature (also ein RT-Feature, welches erst zum Zeitpunkt der Inferenz bekannt ist) verwendet werden.

Beispiel

Ein sehr einfaches Modell mit den geforderten Features könnte die Wahrscheinlichkeit von Regen in den nächsten zwei Stunden vorhersagen (“muss ich einen Schirm einpacken?”). Ein aggregiertes Feature (Batch oder NRT) könnte beispielsweise die durchschnittliche Luftfeuchtigkeit der letzten 24h sein. Ein aktuelles (RT) Feature könnte der Bewölkungsgrad sein.

Schritte bei der Pipeline-Entwicklung

Im folgenden wird der einfachste Aufbau der drei Pipelines mittels Hopsworks grob beschrieben. Es wird empfohlen, möglichst einfach zu beginnen, und nur weiter auszubauen, wenn die einfache Version sauber funktioniert.

1. Feature Pipeline

1. Rohdaten via API von der Quelle abrufen
2. Features engineeren (Rolling Windows, Aggregationen)
3. Dataframe mit Features und Label erstellen
4. Feature Group erstellen mit Primary Key + Event Time
5. Dataframe in die Feature Group einfügen

2. Training Pipeline

1. Feature Group laden
2. Feature View erstellen mit einer Query, die Features + Label auswählt
3. Training Dataset aus der Feature View erstellen
4. Modell trainieren und evaluieren
5. Modell lokal abspeichern (joblib.dump o.ä.), dann in die Model Registry hochladen (create_model, model.save im Falle der Hopsworks Registry)

3. Inference Pipeline

1. Allfällige Live-Daten abrufen oder User-Input erfragen
2. Feature View holen und mit der View die neuesten Features laden
3. Modell aus der Model Registry herunterladen

4. Prediction durchführen

Mögliche Ausbaustufen

- Datensplits im Hopsworks Trainingdataset implementieren anstatt lokal mit Scikit-Learn `train_test_split()`
- Verwenden von mehreren Feature Groups und joinen mittels Key
- Datenvalidierung unter Verwendung der Great Expectations Funktionalität des Featurestores
- Feature Preprocessing mit Hopsworks Transformation functions
- Separieren von Features und Labels für Point-in-time correct joins mittels einer Spine Group
- Zurückspielen von RT-Features (User-Eingaben) und später anfallender Ground Truth in den Featurestore
- Anstatt in einer Pipeline kann die Inferenz auch in einer Webapplikation gemacht werden
- Deployen des Modells in Hopsworks
- Containerisierung

Dokumentation

- Featurestore User Guide
- Featurestore API
- Model Registry User Guide
- Model Registry API

Abgabe und Bewertung

Die Arbeit kann alleine oder in Zweiergruppen gemacht werden

Abgabe

- Die Abgabe erfolgt via Teams Aufgabe
- Abzugeben ist ein Link auf ein Github Repository
 - Das Repository muss den Code der drei Pipelines enthalten (Code einer Applikation anstatt der Inference Pipeline ist zulässig)
 - Ausserdem muss ein Readme.md enthalten sein, welches die verwendeten Features, das Modell sowie den Aufbau der Pipelines beschreibt
 - Falls Schwierigkeiten bestanden und die Pipelines nicht voll funktionsfähig und korrekt sind, müssen diese Schwierigkeiten erläutert werden
- Abgabetermin ist der 21.5.2026 23:59

Bewertung

- Die Bewertung erfolgt als Testat (erfüllt / nicht erfüllt), es gibt keine Benotung

- Beurteilt werden
 - Strukturiertes Vorgehen: Ist die FTI-Architektur sauber umgesetzt und wurde der Featurestore zur Speicherung der Features verwendet?
 - Vollständigkeit der Pipeline: Sind die oben beschriebenen Schritte korrekt umgesetzt? Werden die zwei verlangten Arten von Features berechnet und bei der Inferenz verwendet?
 - Dokumentation und Nachvollziehbarkeit: Sind die verwendeten Daten und das Modell kurz beschrieben? Ist die Arbeit so dokumentiert, dass eine dritte Person die Pipeline verstehen und aufsetzen könnte? Sind Abhängigkeiten (Python Version, installierte Pakete) sauber definiert? Gibt es eine klare Anleitung zum Ausführen?
 - Reflexion und Limitationen: Werden Einschränkungen der gewählten Lösung ehrlich benannt? Gibt es Erläuterungen der Schwierigkeiten, falls die Pipelines nicht voll funktionsfähig und korrekt sind?
- Nicht beurteilt werden
 - Codequalität
 - Modellperformance
 - Allfällig umgesetzte Ausbaustufen